

GPUMODE Lecture Study Notes: Lecture 18

Fusing Kernels

Core Problem the Lecture Addresses

The lecture studies **kernel fusion**: the practice of combining multiple GPU operations into fewer kernels so that intermediate tensors do not repeatedly travel through global memory and so that Python or framework dispatch overhead is reduced.

The main practical question is:

Given a PyTorch model written in eager mode, how can we use profiling, `torch.compile`, generated Triton kernels, and possibly hand-written CUDA to understand and improve performance without immediately rewriting the entire model manually?

The lecture uses two concrete case studies:

1. A recommendation model inspired by **DLRM**, where dense features, sparse embedding features, feature interactions, and a prediction MLP create a realistic optimization target.
2. A small **LoRA-style** model, where low-rank adapter computations introduce extra matrix multiplications and pointwise operations that are natural candidates for fusion.

The recurring theme is that many deep learning models contain sequences such as:

$$Y = \text{ReLU}(XW + b)$$

or

$$Y = \sigma(XW + ABX + b),$$

where the matrix multiplication is expensive but the bias add, activation, scaling, and elementwise transforms are memory-sensitive. Fusion attempts to execute the pointwise parts in one pass over memory.

Required Background

GPU Kernel Launches

A CUDA kernel launch sends work from the CPU host to the GPU device. Each launch has overhead. If a model performs many tiny operations, the GPU may spend too much time launching kernels, synchronizing, and reading or writing intermediate tensors.

A sequence like:

$$A = X + 1, \tag{1}$$

$$B = A \cdot 2, \tag{2}$$

$$C = \text{ReLU}(B) \tag{3}$$

may become three separate kernels in eager PyTorch. Each kernel reads and writes global memory:

$$X \rightarrow A \rightarrow B \rightarrow C.$$

A fused kernel can compute:

$$C = \text{ReLU}(2(X + 1))$$

in one kernel, reading X once and writing C once.

Pointwise Operations

A **pointwise operation** applies independently to each element of a tensor. Examples include addition, multiplication, bias add, ReLU, sigmoid, clamp, and elementwise comparisons.

For a tensor $X \in \mathbb{R}^N$, a pointwise operation has the form:

$$Y_i = f(X_i).$$

Pointwise operations are natural fusion targets because there is no cross-element dependency. Each thread can compute one or more elements independently.

Memory Hierarchy

The relevant GPU memory hierarchy is:

- **Registers:** private per-thread storage, fastest but limited.

- **Shared memory:** on-chip memory shared within a thread block.
- **L1 cache:** close to each SM.
- **L2 cache:** shared across SMs.
- **HBM/global memory:** large but much slower than on-chip storage.

Fusion is often valuable because it avoids materializing intermediate tensors in HBM.

Torch Compile and Torch Inductor

`torch.compile` captures PyTorch programs, lowers them through the compiler stack, and can generate optimized code. In many GPU cases, Torch Inductor emits Triton kernels for fused pointwise operations and sometimes uses CUDA Graphs to reduce launch overhead.

A representative call is:

```
compiled_model = torch.compile(
    model,
    fullgraph=True,
    mode="max-autotune"
)
```

`fullgraph=True` asks PyTorch to compile the whole graph and raise an error if graph breaks occur. The `max-autotune` mode allows more aggressive optimization and may use CUDA Graphs.

Main Concepts

Why Kernel Fusion Helps

Suppose we compute:

$$Y = \text{ReLU}(XW + b).$$

The matrix multiplication produces:

$$Z = XW.$$

Then bias and ReLU are applied:

$$U = Z + b, \quad Y = \max(U, 0).$$

Without fusion, this can involve:

1. One GEMM kernel for $Z = XW$.
2. One pointwise kernel for $U = Z + b$.
3. One pointwise kernel for $Y = \text{ReLU}(U)$.

With fusion, the compiler can keep the elementwise work together:

$$Y_{ij} = \max(Z_{ij} + b_j, 0).$$

This reduces:

- kernel launch overhead,
- global memory traffic,
- intermediate tensor allocation,
- memory allocator pressure,
- synchronization boundaries between operations.

DLRM as the Running Example

DLRM, or Deep Learning Recommendation Model, combines dense numerical features and sparse categorical features.

Dense features may include values such as user age, country encodings, or continuous statistics. Sparse features represent categorical identifiers such as product IDs, ad IDs, user interaction IDs, or hashed categorical values.

The model structure is:

1. Dense features pass through a bottom MLP.
2. Sparse categorical features are mapped through embedding tables.
3. Dense and sparse representations interact.
4. The interaction output passes through a top MLP.
5. A sigmoid output predicts click or interaction probability.

A simplified prediction is:

$$\hat{y} = \sigma(f_{\text{top}}(\text{Interact}(f_{\text{dense}}(x_{\text{dense}}), E(x_{\text{sparse}})))).$$

Here:

- x_{dense} is the dense feature tensor,
- x_{sparse} is the sparse ID tensor,
- $E(\cdot)$ represents embedding lookup,

- f_{dense} and f_{top} are MLPs,
- Interact computes dense-sparse or sparse-sparse feature interactions.

Feature Interaction Explosion

If there are S sparse features and each embedding has dimension D , then interaction cost can grow quickly.

A rough interaction scale is:

$$\text{Interaction Work} \propto B \cdot S^2 \cdot D,$$

where:

- B is batch size,
- S is the number of sparse features,
- D is embedding dimension.

Second-order interactions are expensive because every feature can interact with many other features. The top MLP may become compute-heavy because the interaction vector becomes wide.

Sparse Features and Embedding Bandwidth

Sparse features are often memory-bandwidth sensitive. An embedding lookup fetches rows from an embedding table:

$$e_i = E[\text{id}_i],$$

where $E \in \mathbb{R}^{C \times D}$, C is cardinality, and D is embedding dimension.

The larger the embedding table and the more random the IDs, the harder it is to obtain cache locality. Sparse embedding workloads frequently stress memory capacity and memory bandwidth rather than raw FLOPs.

Hardware-Level Explanation

Eager PyTorch Execution

In eager mode, each operation is executed immediately. This is excellent for usability and debugging but can be inefficient for many small GPU operations.

A sequence such as:

```

x = linear(input)
x = x + bias
x = torch.relu(x)
x = x * scale

```

may produce multiple GPU kernels. Each kernel has to read and write global memory. The GPU may not reach high utilization if each operation is small.

Kernel Fusion on the GPU

A fused pointwise kernel maps tensor elements to GPU threads. Each thread may do:

```

float z = input[idx];
z = z + bias[col];
z = max(z, 0.0f);
z = z * scale;
output[idx] = z;

```

The important hardware behavior is:

- each thread computes one or more tensor elements,
- reads are usually coalesced if adjacent threads read adjacent elements,
- intermediate values stay in registers,
- only the final output is written to global memory.

CUDA Graphs

CUDA Graphs reduce CPU launch overhead by capturing a sequence of GPU operations and replaying them. In the lecture, `torch.compile` with modes such as `max-autotune` or `reduce-overhead` can use CUDA Graphs internally.

The simplified idea is:

Many Launches $\rightarrow$ Captured Graph $\rightarrow$ Replay Graph.

This helps especially when the sequence of operations and tensor shapes are static. Dynamic shapes are harder because the graph may need to be specialized for multiple shape cases.

Static Shapes and Dynamic Shapes

Static shapes help compilers generate specialized code. If the compiler knows that a tensor has shape 128×512 , it can hard-code sizes, strides, and launch parameters.

Dynamic shapes require more general code or multiple compiled variants:

$$\{s_1, s_2, \dots, s_k\} \Rightarrow \{\text{compiled graph for } s_1, \dots, \text{compiled graph for } s_k\}.$$

Padding can force static shapes, but padding adds memory and compute overhead. Nested or jagged tensors are a more natural representation for variable-length data, but they are more challenging for compilers.

Mathematical Reasoning

Arithmetic Intensity

Arithmetic intensity is:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes Moved}}.$$

A kernel is more likely to be memory-bound when arithmetic intensity is low. Pointwise operations usually perform few FLOPs per byte loaded.

For a simple ReLU:

$$y_i = \max(x_i, 0),$$

each element requires approximately one comparison, one read, and one write. For FP32:

$$\text{Bytes Moved} \approx 4 + 4 = 8 \text{ bytes.}$$

The arithmetic intensity is tiny:

$$\text{AI}_{\text{ReLU}} \approx \frac{1}{8}.$$

Fusion improves effective arithmetic intensity by doing several operations per read and write.

For:

$$y_i = \max(\alpha x_i + \beta, 0),$$

there are more FLOPs but still only one read and one write if fused. Therefore:

$$\text{AI}_{\text{fused}} > \text{AI}_{\text{separate}}.$$

GEMM Cost

For a matrix multiplication:

$$C = AB,$$

with:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N},$$

the FLOP count is approximately:

$$\text{FLOPs} = 2MKN.$$

GEMM is often compute-heavy and can use tensor cores when dtype and layout permit. In contrast, bias add and activation are bandwidth-sensitive. This is why compilers often leave GEMM as a library or generated matmul call and fuse the pointwise operations around it.

Bias Broadcasting via Modular Indexing

For a tensor $X \in \mathbb{R}^{B \times H}$ and bias $b \in \mathbb{R}^H$, the bias add is:

$$Y_{ij} = X_{ij} + b_j.$$

If the tensor is flattened into one dimension, an element index p maps to column:

$$j = p \bmod H.$$

This is why generated Triton kernels often contain modular arithmetic such as:

$$j = p \bmod 512.$$

It allows each element to fetch the correct bias value while processing a flattened memory region.

Code Walkthrough

Representative DLRM Forward Pass

A simplified version of the DLRM-style model is:

```

class DLRM(torch.nn.Module):
    def __init__(self, dense_mlp, sparse_embeddings, top_mlp):
        super().__init__()
        self.dense_mlp = dense_mlp
        self.sparse_embeddings = sparse_embeddings
        self.top_mlp = top_mlp

    def forward(self, dense_features, sparse_ids):
        dense_out = self.dense_mlp(dense_features)

        sparse_vectors = []
        for table_id, embedding_table in enumerate(self.sparse_embeddings):
            ids = sparse_ids[:, table_id]
            sparse_vectors.append(embedding_table(ids))

        sparse_out = torch.stack(sparse_vectors, dim=1)
        interaction = self.interact(dense_out, sparse_out)

        logits = self.top_mlp(interaction)
        prediction = torch.sigmoid(logits)
        return prediction

```

The dense MLP performs matrix multiplications and activations. The sparse path performs embedding lookups. The interaction stage can create a wide tensor, and the top MLP can become compute-intensive.

Bad Hashing Path

The lecture first shows an intentionally inefficient path where sparse ID tokenization or hashing happens during every forward pass and causes excessive copies.

```

def forward_index_hash(self, sparse_features):
    mapped_features = []
    for feature_id in range(sparse_features.shape[1]):
        mapping = self.index_hash_maps[feature_id]
        values = sparse_features[:, feature_id]
        mapped = torch.tensor(
            [mapping[int(v)] for v in values],
            device=sparse_features.device
        )
        mapped_features.append(mapped)

    return torch.stack(mapped_features, dim=1)

```

This is inefficient because it mixes Python loops, tensor construction, and device copies inside the forward path. Profiling shows excessive time in hashing and

copying instead of model computation.

Improved Modular Hashing

A more GPU-friendly simplification is modular hashing:

$$\text{id}_{\text{hashed}} = \text{id} \bmod C,$$

where C is the feature cardinality.

```
def forward_mod_hash(self, sparse_features):  
    return sparse_features % self.cardinalities
```

This is simple, vectorized, and can run efficiently on the GPU. It removes repeated CPU-side mapping and reduces copy overhead.

Using Torch Compile

The lecture then applies `torch.compile`:

```
model = DLRM(...)  
compiled_model = torch.compile(  
    model,  
    fullgraph=True,  
    mode="max-autotune"  
)  
  
out = compiled_model(dense_features, sparse_features)
```

This can produce:

- CUDA Graph launches,
- generated Triton kernels,
- fused pointwise operations,
- reduced memory allocations,
- reduced Python dispatch overhead.

Representative Generated Triton Pointwise Kernel

A simplified Triton-style fused bias and ReLU kernel is:

```
@triton.jit  
def fused_bias_relu(x_ptr, bias_ptr, out_ptr, n_elements: tl.constexpr, hidden: tl.constexpr):  
    pid = tl.program_id(0)  
    offsets = pid * BLOCK + tl.arange(0, BLOCK)  
    mask = offsets < n_elements
```

```

x = tl.load(x_ptr + offsets, mask=mask)
cols = offsets % hidden
b = tl.load(bias_ptr + cols, mask=mask)

y = x + b
y = tl.maximum(y, 0.0)

tl.store(out_ptr + offsets, y, mask=mask)

```

The important details are:

- `offsets` defines the flattened element positions handled by one Triton program.
- `cols = offsets % hidden` implements broadcasting of the bias vector.
- `tl.load` reads input and bias.
- `tl.maximum` implements ReLU.
- `tl.store` writes only the final result.

This avoids writing the intermediate $X + b$ tensor to HBM.

Equivalent Naive CUDA Kernel

A direct CUDA equivalent is:

```

__global__ void fused_bias_relu_cuda(
    const float* x,
    const float* bias,
    float* out,
    int n_elements,
    int hidden
) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < n_elements) {
        int col = idx % hidden;
        float y = x[idx] + bias[col];
        y = y > 0.0f ? y : 0.0f;
        out[idx] = y;
    }
}

```

This kernel is conceptually correct, but the lecture notes that naive CUDA generated from Triton is not automatically optimal. Triton benefits from compiler lowering and optimization. Hand-written CUDA may require more tuning around block size, vectorization, occupancy, memory access, and register use.

LoRA Fusion Case Study

LoRA Computation

LoRA fine-tunes a model by freezing the original weight matrix and learning a low-rank update.

For a linear layer:

$$Y = XW,$$

LoRA modifies it as:

$$Y = XW + \alpha XAB,$$

where:

$$A \in \mathbb{R}^{d_{\text{in}} \times r}, \quad B \in \mathbb{R}^{r \times d_{\text{out}}}, \quad r \ll \min(d_{\text{in}}, d_{\text{out}}).$$

The rank r is small, so the trainable parameter count is reduced from:

$$d_{\text{in}}d_{\text{out}}$$

to:

$$r(d_{\text{in}} + d_{\text{out}}).$$

Representative LoRA Layer

```
class LoRALinear(torch.nn.Module):
    def __init__(self, in_features, out_features, rank, alpha):
        super().__init__()
        self.weight = torch.nn.Parameter(
            torch.randn(in_features, out_features),
            requires_grad=False
        )
        self.A = torch.nn.Parameter(torch.randn(in_features, rank))
        self.B = torch.nn.Parameter(torch.randn(rank, out_features))
        self.alpha = alpha

    def forward(self, x):
        base = x @ self.weight
        update = (x @ self.A) @ self.B
        return base + self.alpha * update
```

The unfused computation has several stages:

$$Y_{\text{base}} = XW, \quad (4)$$

$$H_{\text{lora}} = XA, \quad (5)$$

$$Y_{\text{lora}} = H_{\text{lora}}B, \quad (6)$$

$$Y = Y_{\text{base}} + \alpha Y_{\text{lora}}. \quad (7)$$

The final add and scaling are pointwise and are natural fusion targets.

Fusing LoRA Add, Scale, and Activation

If an activation is applied:

$$Z = \phi(Y_{\text{base}} + \alpha Y_{\text{lora}} + b),$$

then a compiler can fuse:

$$z_i = \phi(y_{\text{base},i} + \alpha y_{\text{lora},i} + b_i).$$

A representative fused kernel is:

```
@triton.jit
def fused_lora_activation(base_ptr, lora_ptr, bias_ptr, out_ptr, n_elements: tl.constexpr, h
    pid = tl.program_id(0)
    offsets = pid * BLOCK + tl.arange(0, BLOCK)
    mask = offsets < n_elements

    base = tl.load(base_ptr + offsets, mask=mask)
    lora = tl.load(lora_ptr + offsets, mask=mask)
    cols = offsets % hidden
    bias = tl.load(bias_ptr + cols, mask=mask)

    y = base + alpha * lora + bias
    y = 1.0 / (1.0 + tl.exp(-y))

    tl.store(out_ptr + offsets, y, mask=mask)
```

This kernel fuses add, multiply, bias add, and sigmoid. It does not fuse the matrix multiplications themselves.

Performance Intuition

Why Fusing Everything Is Not Always Good

A common misconception is that one giant kernel is always fastest. This is not true.

A mega-kernel may suffer from:

- excessive register pressure,
- register spilling into local memory,
- reduced occupancy,
- complicated control flow,
- poor maintainability,
- lower compiler optimization quality,
- difficulty reusing library-level GEMM optimizations.

If each thread needs many live variables, register usage per thread increases. Occupancy can decrease because each SM can hold fewer active warps.

A simplified occupancy constraint is:

$$\text{Active Threads Per SM} \leq \frac{\text{Registers Per SM}}{\text{Registers Per Thread}}.$$

As registers per thread increase, active warps may decrease. This can reduce latency hiding.

Good Fusion Targets

Good fusion targets usually include:

- bias add,
- activation,
- scaling,
- dropout-like masks,
- residual adds,
- normalization epilogues,
- simple elementwise transforms.

These operations are bandwidth-heavy and have simple per-element logic.

Bad or Difficult Fusion Targets

Fusion becomes harder when operations require:

- global synchronization,
- reductions across large dimensions,
- complex dynamic control flow,
- data-dependent shapes,
- many temporary values,
- complicated memory layouts,
- operations already heavily optimized by cuBLAS, cuDNN, or CUTLASS.

GEMM itself is usually better handled by highly optimized matmul kernels unless one is writing a specialized fused epilogue or a full custom kernel.

Memory Allocation Improvements

Fusion can reduce the memory profile from repeated allocate-deallocate patterns to fewer persistent buffers. Instead of:

$$X \rightarrow T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow Y,$$

the fused path becomes:

$$X \rightarrow Y.$$

This reduces pressure on the allocator and lowers peak memory.

Common Misconceptions

Misconception: Torch Compile Is Only a Black Box

`torch.compile` can be used as a learning tool. The generated code can be dumped, inspected, copied, simplified, and used to understand what kinds of operations the compiler fused.

Misconception: Triton Code Is Always Equivalent to CUDA Code

Triton and CUDA expose different abstraction levels. A naive CUDA translation of Triton may be correct but slower. Triton relies on its compiler stack to generate efficient GPU code.

Misconception: Static Shapes Are Just a Minor Detail

Static shapes are a major optimization enabler. They allow hard-coded dimensions, better graph capture, and more specialized kernels. Dynamic shapes are useful but usually harder to optimize.

Misconception: Fusion Means Combining the Entire Model

Fusion is usually local and selective. The most common pattern is fusing pointwise operations around expensive kernels, not turning the entire model into one enormous kernel.

Misconception: Profiler Times Are Always Obvious

Operations such as `item()` can introduce synchronization. A CPU read from a GPU tensor forces the CPU to wait for the GPU to finish. This can distort profiler traces if not understood.

Practical Takeaways

- Start with profiling before writing kernels.
- Look for excessive copies, synchronizations, small kernels, and repeated allocations.
- Keep data on the GPU when possible.
- Avoid Python loops and tensor construction inside hot forward paths.
- Use vectorized GPU operations before writing custom kernels.
- Try `torch.compile` early.
- Inspect generated Triton code to learn what the compiler fused.
- Treat generated kernels as educational artifacts, not always final production code.
- Fuse pointwise operations when they share the same shape and access pattern.
- Be cautious about mega-kernels because register pressure and maintainability can dominate.

Project or Experiment Ideas

1. Implement a small MLP in eager PyTorch and profile every operation.
2. Add `torch.compile` and compare kernel counts, memory allocations, and runtime.

3. Dump Torch Inductor generated Triton code and identify pointwise fusion kernels.
4. Write a manual Triton fused bias-ReLU kernel and compare it against eager PyTorch.
5. Translate the Triton kernel to CUDA and benchmark the naive CUDA version.
6. Test how batch size affects the benefit of fusion.
7. Compare static-shape and dynamic-shape versions of the same model.
8. Build a small LoRA linear layer and examine whether scaling, add, and activation are fused.
9. Use Nsight Systems or PyTorch Profiler to detect `item()` synchronization.
10. Measure peak memory with and without `torch.compile`.

Final Mental Model

Kernel fusion is best understood as **keeping temporary values close to the computation**. Instead of launching many kernels that repeatedly read and write HBM, a fused kernel reads inputs once, performs several simple operations in registers, and writes the final output once.

For modern PyTorch workflows, the practical path is:

Eager Model → Profile → `torch.compile` → Inspect Generated Triton → Decide Whether Manual Triton or

The engineer's job is not to fuse everything blindly. The engineer's job is to identify where memory traffic, launch overhead, and unnecessary intermediate tensors are dominating, then fuse only the operations whose access patterns and hardware behavior make fusion profitable.